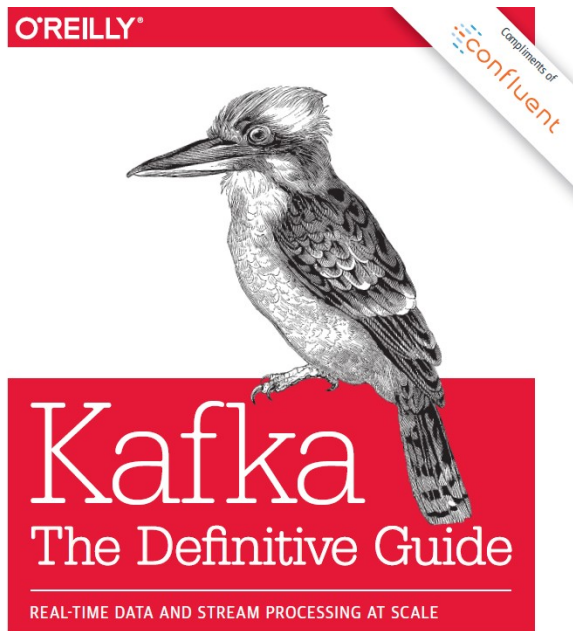


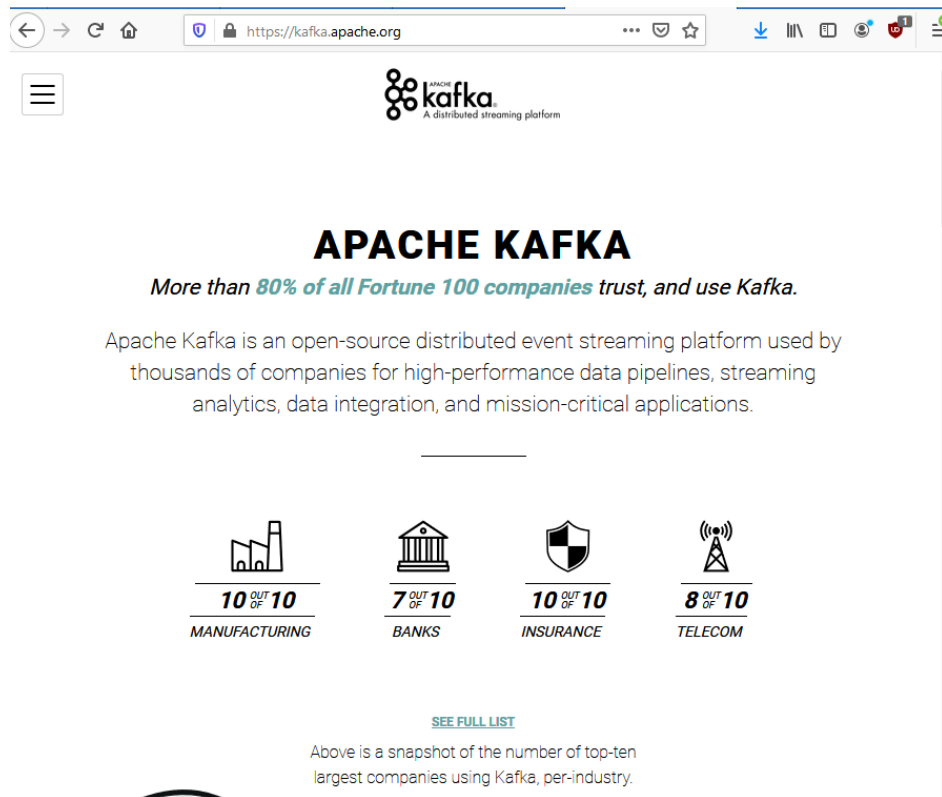


integrata
cegos

Apache Kafka



Neha Narkhede,
Gwen Shapira & Todd Palino



The screenshot shows the Apache Kafka website. The header includes the Apache Kafka logo and the tagline 'A distributed streaming platform'. The main heading is 'APACHE KAFKA', followed by the text 'More than 80% of all Fortune 100 companies trust, and use Kafka.' Below this, a paragraph describes Apache Kafka as an open-source distributed event streaming platform. A horizontal line separates this from a section showing industry adoption statistics. These statistics are presented as icons with 'X OUT OF 10' ratings: Manufacturing (10/10), Banks (7/10), Insurance (10/10), and Telecom (8/10). A link 'SEE FULL LIST' is provided. At the bottom, a note states: 'Above is a snapshot of the number of top-ten largest companies using Kafka, per-industry.'

Industry	Rating
MANUFACTURING	10 OUT OF 10
BANKS	7 OUT OF 10
INSURANCE	10 OUT OF 10
TELECOM	8 OUT OF 10

- The tools and frameworks used in this seminar are open source
 - LPGL license model
- This is a programming seminar
 - This means that the content is deepened and internalized through exercises
 - Sample examples are provided
- Documentation and resources are also available online
- Conventions
 - Commands are displayed in `Courier font`
 - presented in *cursive Courier font*
 - Links appear in `underlined Courier font`

© Javacream

Java cream

Dr. Rainer Sawitzki

Alois- Gilg -Weg 6

81373 Munich

All rights reserved, including those of extracts, photomechanical and electronic reproduction.

introduction	6
Realization of a scaling system	16
Setting up a Kafka system	34
How Kafka works	43
Topics	65
Programming with Kafka	95
Administration and monitoring	109
Kafka extensions	119

1

INTRODUCTION

1.1

CURRENT SITUATION

- Host computers on which a wide variety of products/servers run
 - Database systems
 - Relational databases
 - also NoSQL environment
 - server
 - especially web servers
- network
 - TCP/IP
 - Alignment with http/https, keyword RESTful Web Services
- Local resources on each host
 - File system
- Trend towards services and microservices architectures
 - Today, a specialist application consists of a large number of independent processes

- Each of these systems produces a whole lot of (log) information
 - Distribution to the respective target systems via well-defined communication channels
 - Accessing the local resources of the host computers is often very time-consuming
 - Central joint control/monitoring is necessary
- The specialist processes also communicate intensively with each other
 - With different guarantees of message transmission
 - “Fire and Forget” for Heartbeats
 - “At Least Once ” for update news
 - “Exactly Once ” for consistent data storage

- Central logging
 - Elasticsearch
 - Splunk
 - ...
- Communication via messaging systems or event bus
 - WebSphere MQ
 - ActiveMQ
 - ...

- A valid question!
 - “Kafka doesn’t solve new problems”
 - “Kafka is not without competition”
 - “Instead of a new system, we rely on proven solutions”
- But Kafka is definitely a good product
 - Modern, scalable cluster architecture with low administration effort
 - Available and fail-safe data storage even in high-load areas
 - Simple programming model with drivers for common programming languages
 - With Kafka Connect and Kafka Streams, other powerful libraries are available for integration and data analysis

1.2

FEATURES OF APACHE KAFKA

- The main focus is to write down information
 - Potentially infinitely large
 - Scalability must be guaranteed
 - Dynamic, no maintenance windows etc. necessary
 - Potentially any “broadband”
 - The limit of data throughput is the network
- Here Kafka behaves like a normal database system

- The data is actively distributed to registered consumers
 - Management logic through algorithms
 - Kafka uses and requires CPU resources
- Kafka behaves like a messaging system here

- Streaming in Kafka
 - Filter
 - transformation
 - Aggregate
- Kafka behaves like an event bus here

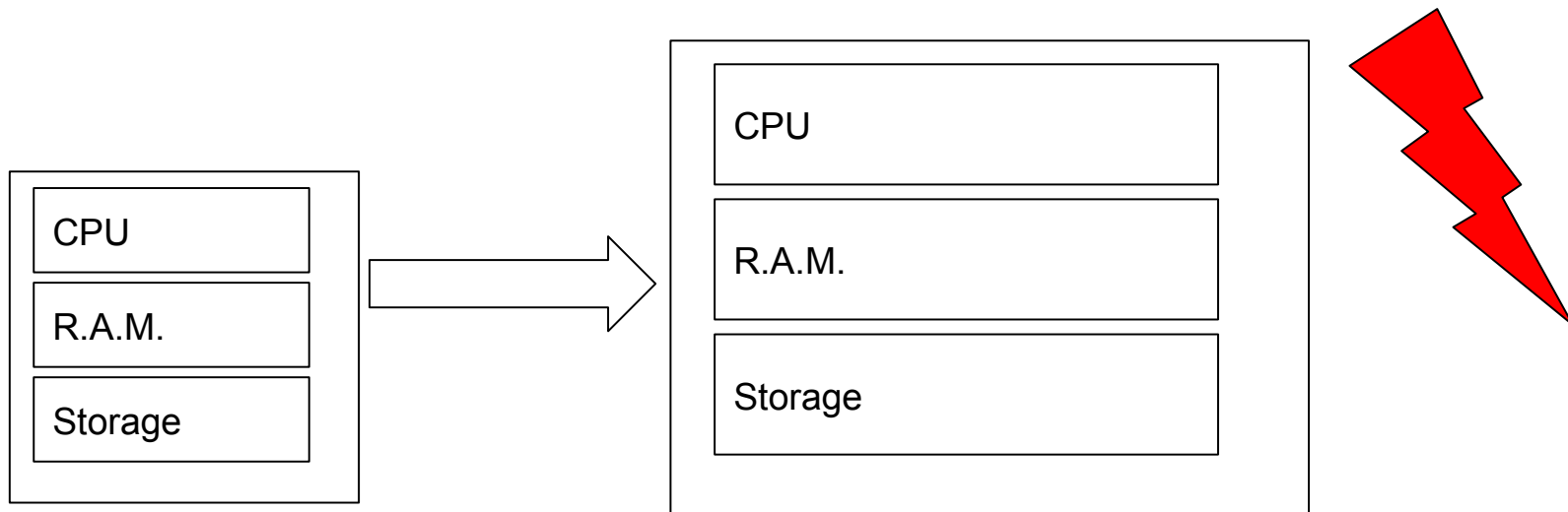
2

REALIZATION OF A SCALING SYSTEM

2.1

CLUSTER APPROACHES

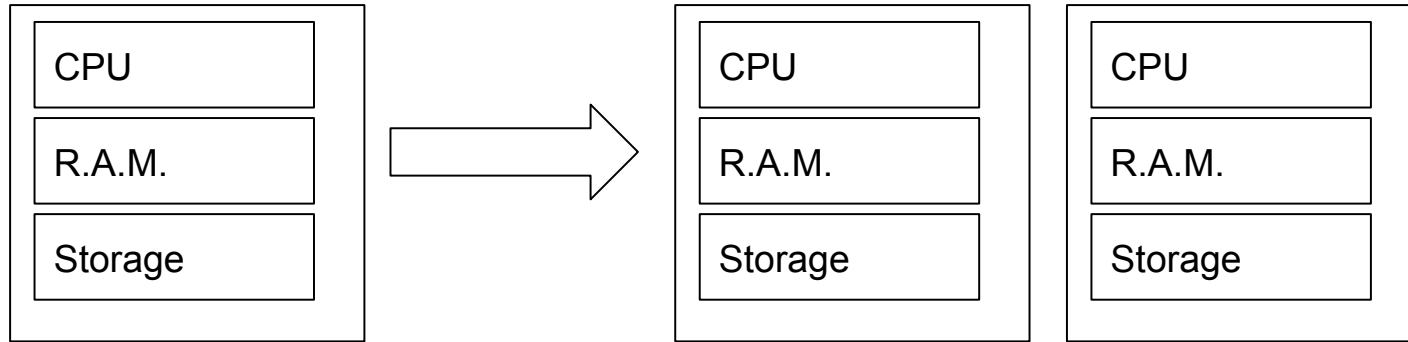
- Limits in CPU/RAM/Storage are resolved by migrating to a larger system
- Consistent data storage can be achieved relatively



Second option: horizontal scaling

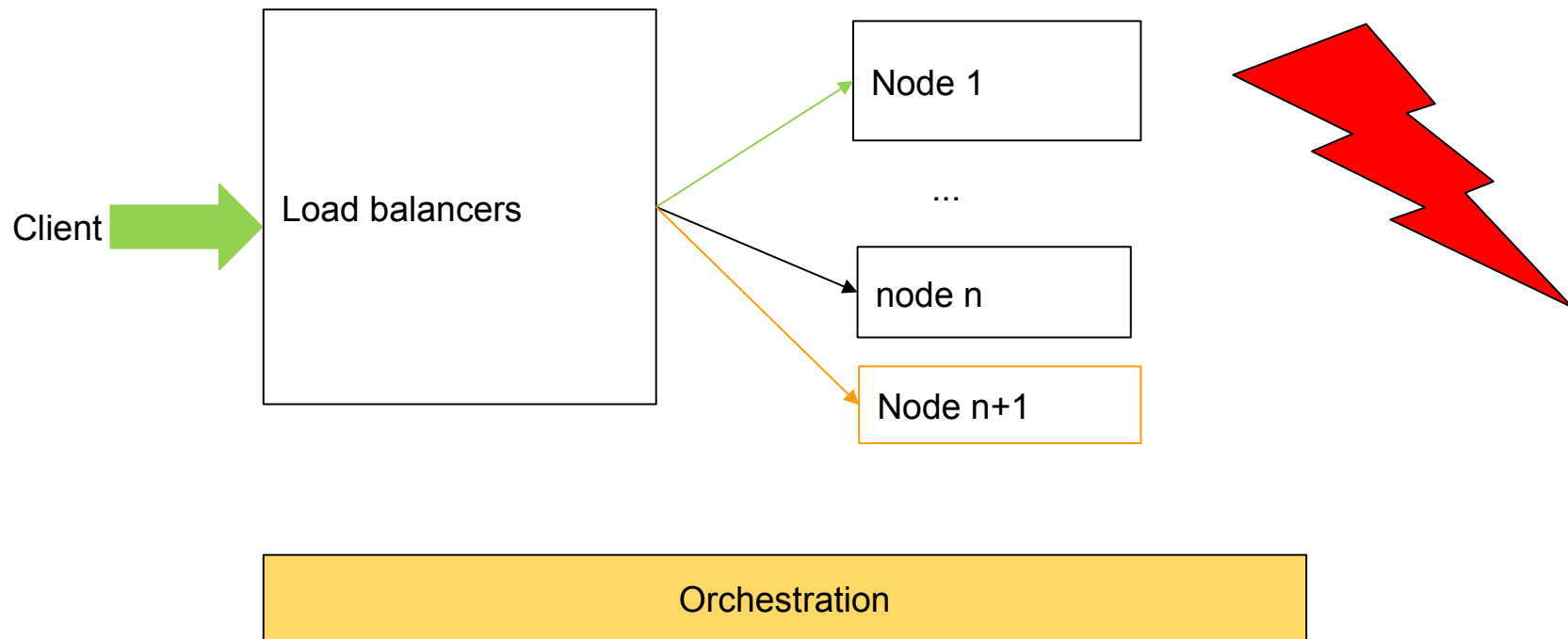
- Scaling by dynamically adding (up-scaling) or removing (down-scaling) brokers
- Data consistency is more complex to achieve here

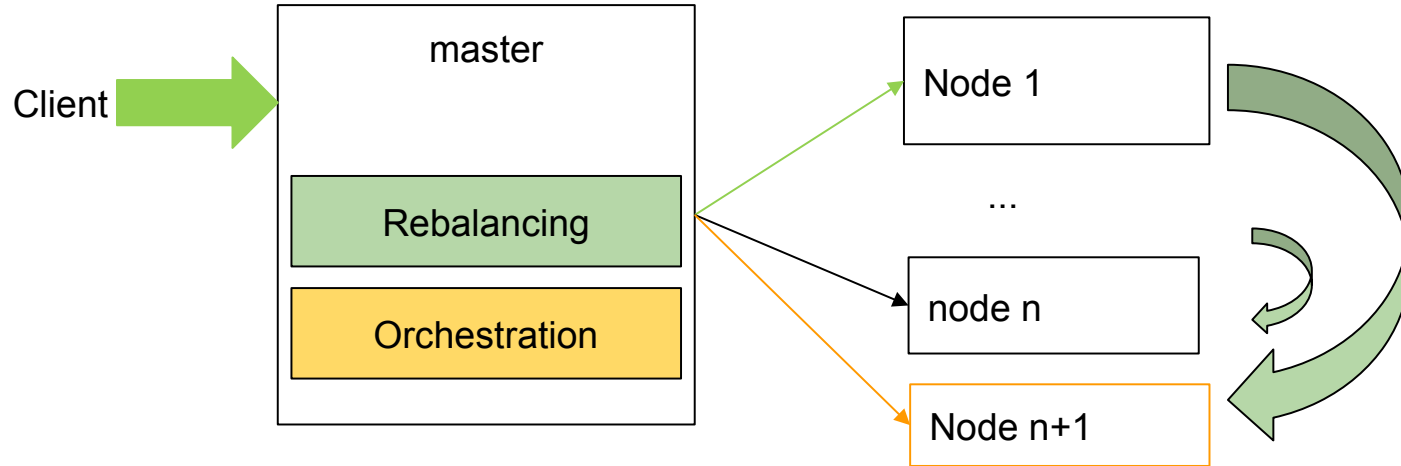
Second option: horizontal scaling

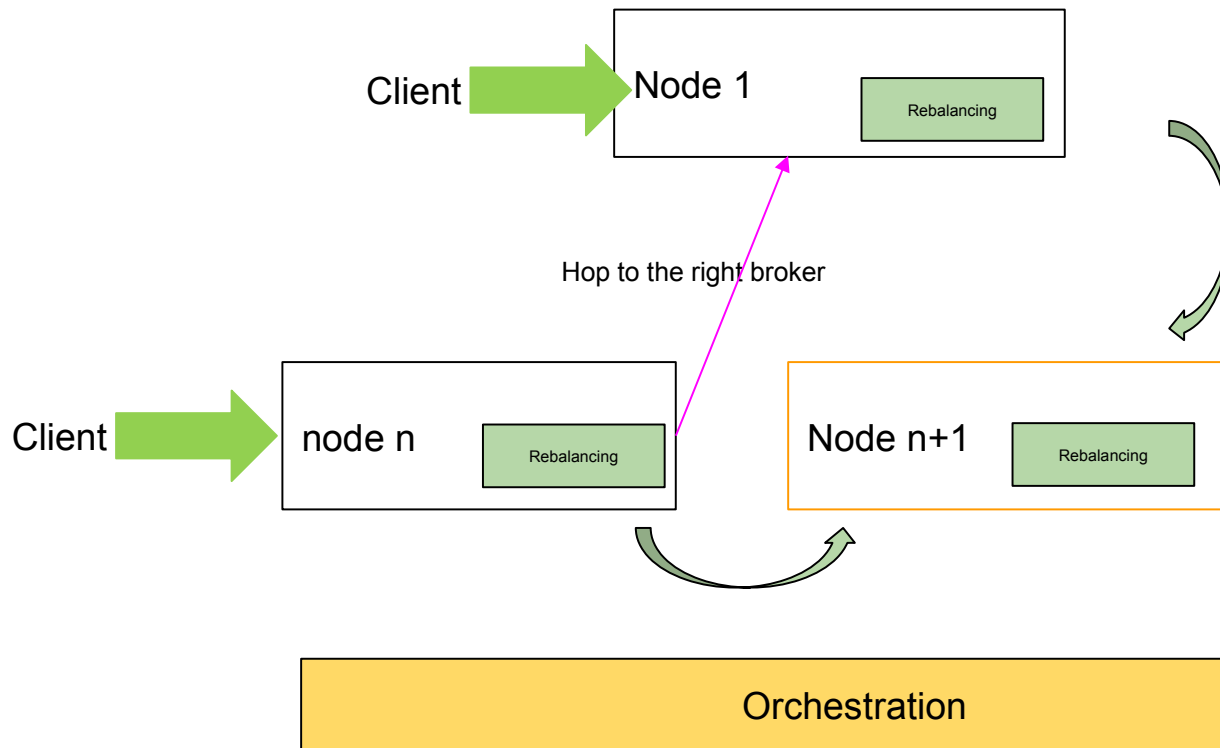


Kafka is a horizontally scaling system

- Orchestration of stateless brokers
- Cluster with master
- Masterless Ring Clusters







- Easy to implement from an operational perspective:
 - Load balancer combined with orchestration
 - Masterless Ring Clusters
- A master makes implementation easier for the broker manufacturer
 - Data consistency

- Load balancing and orchestration
 - Any microservices architecture
- Master based cluster
 - Database systems
 - Postgres
 - MongoDB
- Ring clusters
 - Database systems
 - Apache Cassandra
 - Couchbase
 - Apache Ignite

2.2

KAFKA BROKERS

- master
 - Apache Zookeeper
 - Storage of the cluster state
 - Broker State
 - Not the actual data storage!
 - Locks
 - Other features of Zookeeper are not used by Kafka
 - Kafka brokers also communicate in a ring architecture
 - Forwarding client requests to the “right” broker
 - ...

Why both variants in one product?

- Master broker is easier to implement than a ring when data consistency is required
- But Masterless Ring is much easier to administer
 - “admin-less”
- Kafka “work in progress”
 - The first Kafka versions were clearly master brokers
 - With each release the use of Zookeeper was reduced
 - Kafka is planned as a complete ring cluster

- **Controllers**
 - A preferred broker that controls the entire ring
 - Is “chosen” by all brokers
- **leader**
 - The active data sink/data source

- master
 - Apache Zookeeper
 - Collects information about the status of each broker instance
 - Restarted or stopping brokers communicate with Zookeeper
 - Every broker can store information in Zookeeper
 - Zookeeper is, somewhat simplified, a hierarchical file system
 - As a single point of Failure is operated as an ensemble in Zookeeper in real operation
- The functioning of the ring cluster is also visible
 - A client program can contact a broker node for specific actions
 - Rebalancing of the nodes is initiated by Zookeeper but implemented by the nodes

- More precisely: “Why does Kafka **currently work** like this?”
- Work in progress
 - Originally, the zookeeper was a real master
 - all client communication went through it
 - Refactoring Kafka to a pure ring cluster
 - “Zookeeper wants to be removed in future versions ...”

3

SETTING UP A KAFKA SYSTEM

3.1

SYSTEM REQUIREMENTS

- Java Runtime
 - This makes Apache Kafka platform independent
 - Java 8 or greater
- At least Java processes required:
 - Zookeeper
 - At least one Kafka broker
- Hardware requirements are moderate
 - Zookeeper: 256MB RAM
 - Kafka broker 1GB RAM (512MB is also possible)

- Docker images with Kafka are possible
- Apache Kafka Community does not provide a finished Docker image
 - Wurstmeister
 - Bitnami,
 - Spotify
 - Confluent
 - Commercial Kafka solution provider

3.2

DOWNLOAD

- Platform-independent download from the Apache pages
- *am*
 - Shell scripts and Windows Bat
- *config*
 - Platform independent
- Caution
 - In addition to the essential files, there are also many examples etc
 - Stripped down
 - Start scripts for Zookeeper and Kafka Broker
 - Configuration *zookeeper.properties* and *server.properties*

3.3

DOWNLOAD

- **Zookeeper**

- `bin/ zookeeper-server-start.sh|bat / config / zookeeper.properties`
 - From now on: Java process listening on port 2181

- **Kafka brokers**

- `bin/ kafka-server-start.sh|bat / config / server.properties`
 - From now on: Java process listening on port 9092

- **Potential problems**

- “Java not found ”
 - Java installed? CHECK java version
- “Can’t find path”
 - CHECK: Path to Java Runtime with spaces under Windows?
- “Path too long” or something like that
 - Workaround for Windows:

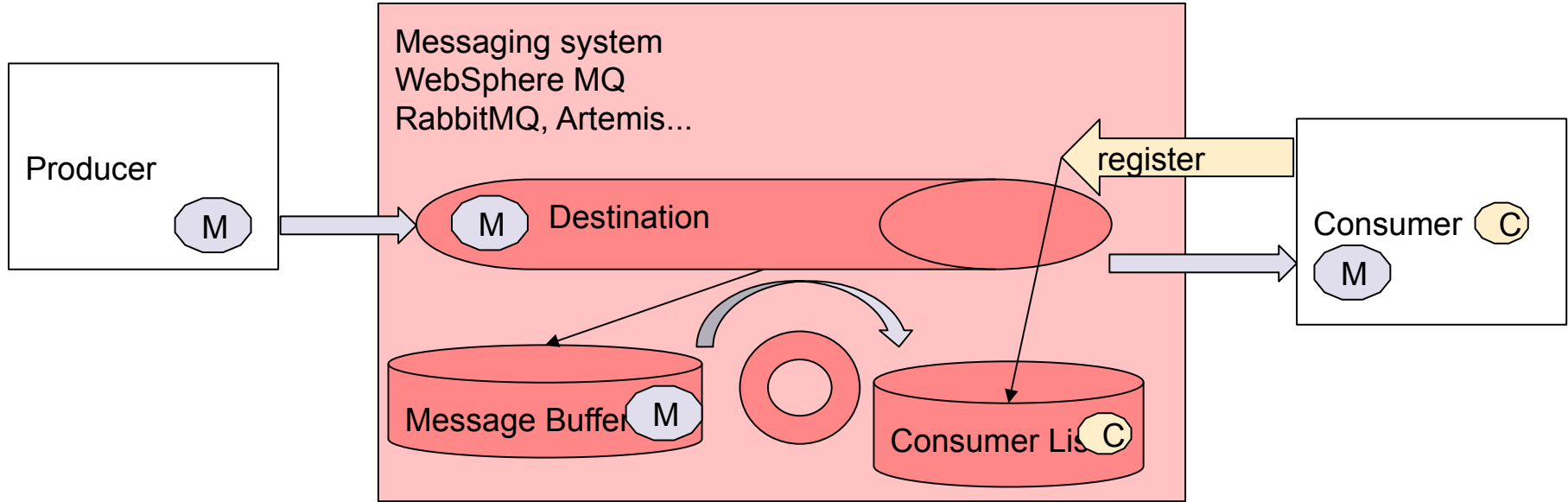
- Linux
 - Producer
 - `kafka-console-producer.sh --broker-list localhost:9092 --topic demo`
 - Consumer
 - `kafka-console-consumer.sh--bootstrap-server localhost:9092 --topic demo`
- Windows
 - Corresponding scripts under `bin\ windows`
 - Alternatively: Git Bash or similar

4

HOW KAFKA WORKS

4.1

ANALOGY TO MESSAGING SYSTEMS



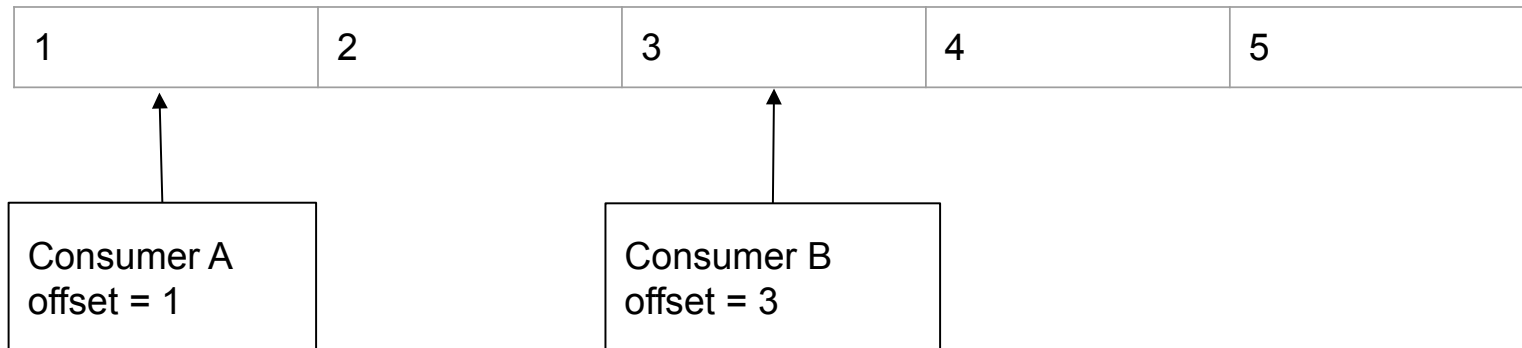
- Message Buffer
 - filled when a message arrives
 - After successful delivery, the message must be deleted again
- Storage of the message as a file in the file system or in a database
- From the producer's perspective, the messaging system will acknowledge receipt of the message as soon as it has been saved in the message buffer

- Delivery strategies
 - “Publish/Subscribe”
 - The message is delivered to all registered and active consumers
 - and then deleted from the message buffer
 - “Point to Point”
 - The message is held until exactly one active consumer receives it
 - and then deleted from the message buffer
 - “Publish/Durable Subscriptions”
 - The message is delivered to all registered consumers and is therefore held until the last registered consumer became active again
 - and then deleted from the message buffer

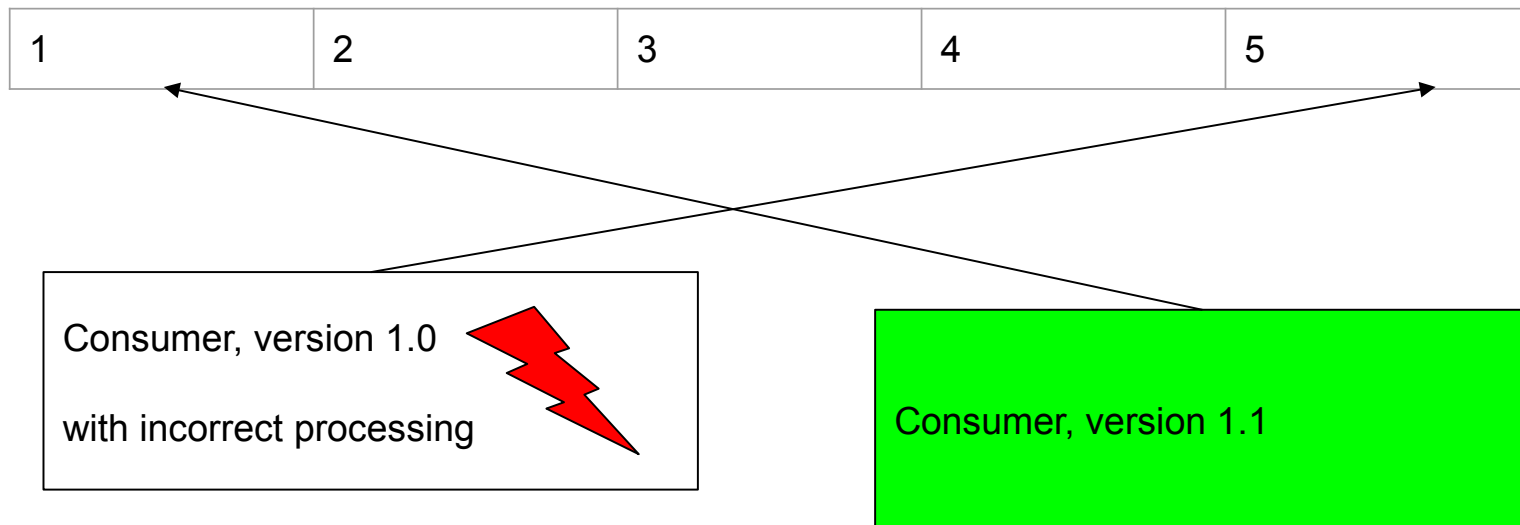
4.2

KAFKA AS A MESSAGING SYSTEM

- “Kafka doesn’t delete at all”
 - Message Buffer is an “ append-only log”
 - Automatic deletion due to timeout (e.g. 30 days) or size (e.g. 500 GB)
- This makes Kafka behave like a database (!)
- A consumer can also view messages that have already been read/delivered at any time
 - Kafka must maintain a “state” for each consumer process
 - “Offset” per consumer session



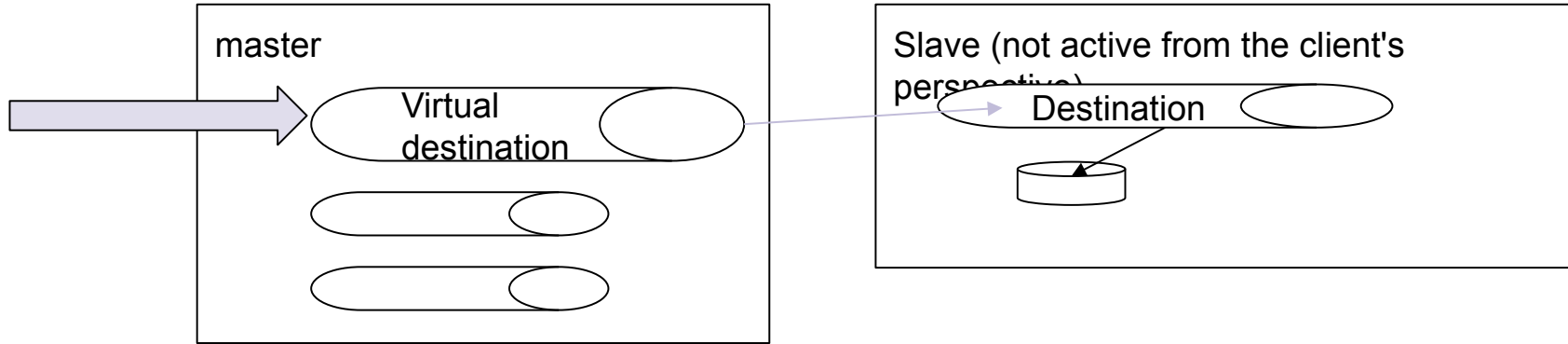
Example: BugFix of a consumer

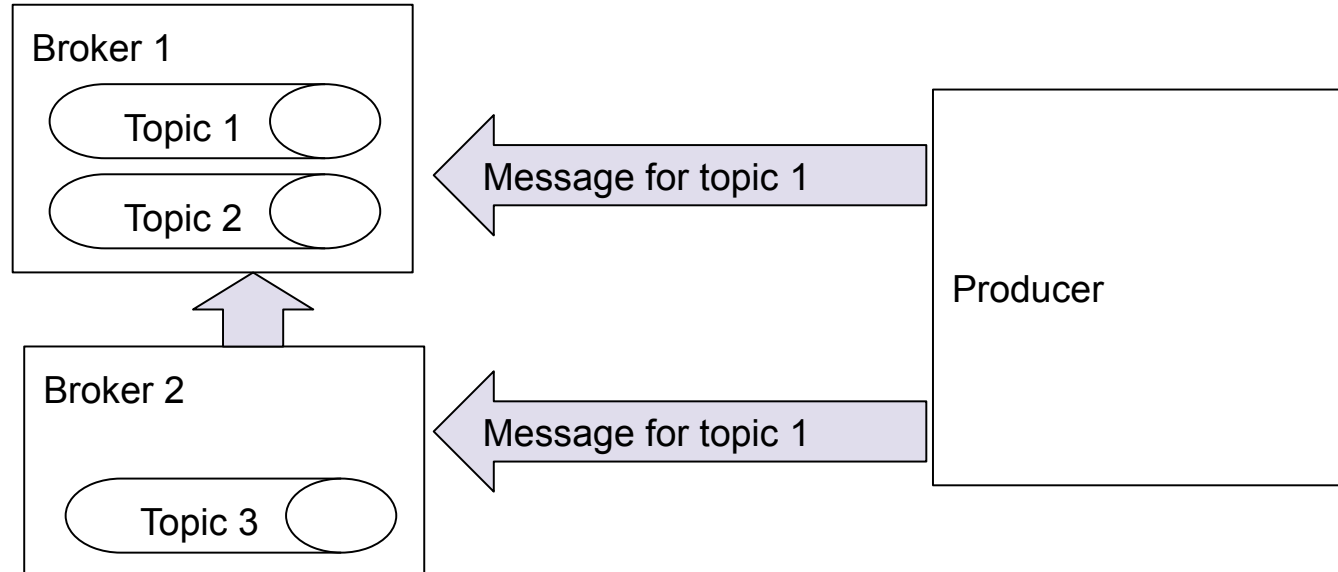


4.3

SCALING

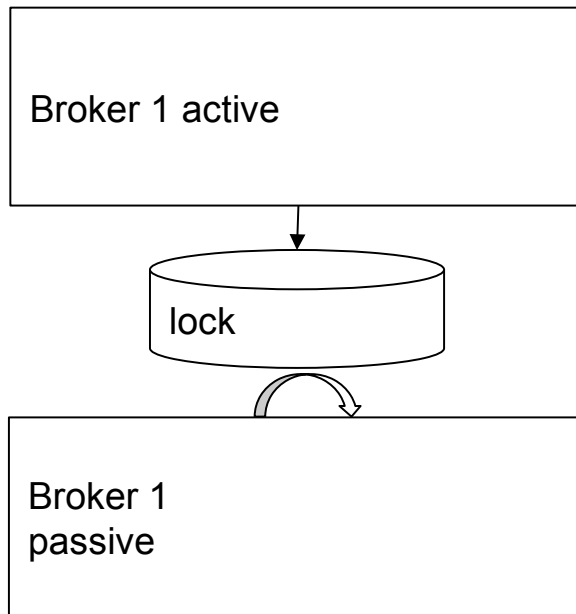
- Delivery guarantees are very difficult to implement in the cluster
 - a really big problem
- In practice, only one broker is actually the active master

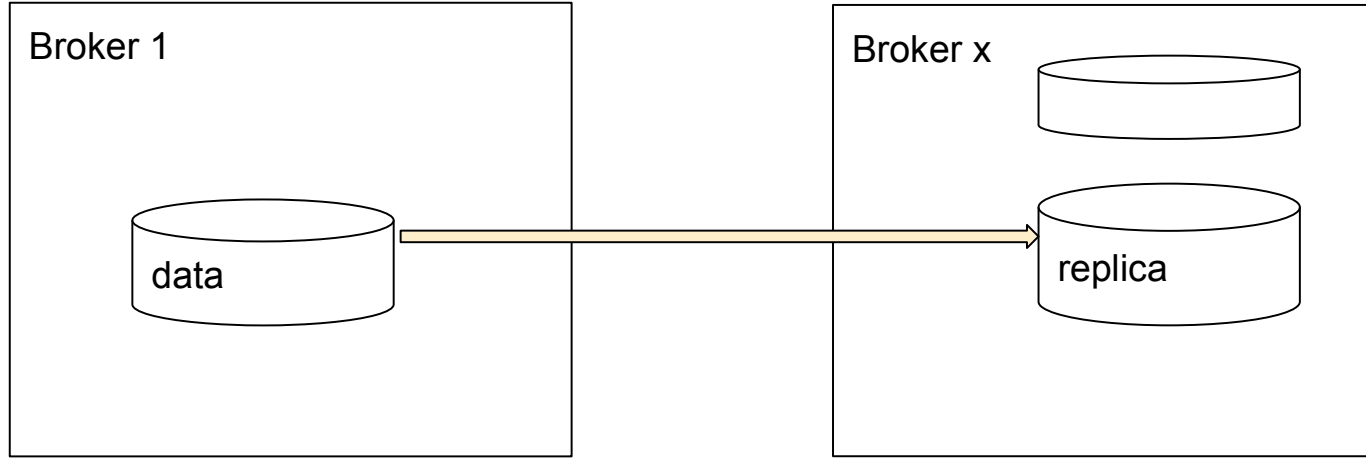




4.4

RESILIENCY





- Topic level replication
- The active broker of a topic is called “Leader”
- Notice
 - Complete reliability cannot be guaranteed without effort
 - When a new message arrives, how many replication servers must be synchronized before the producer receives a successful message delivery signal?

- Knows all active brokers
- Also knows all the current leaders
 - locking ” for every topic
- Saving the current offsets of the consumers
- Caution:
 - Work in progress
 - New Kafka releases increasingly avoid data storage in Zookeeper

- Problem that affects all database systems
- CAP theorem
 - Consistency
 - Availability
 - Partition Tolerance (Can network failures between broker nodes be tolerated?)
 - “Only two out of three possible”
- Kafka must follow the CAP theorem
 - In-Sync Replication (ISR)
 - Out of sync replication

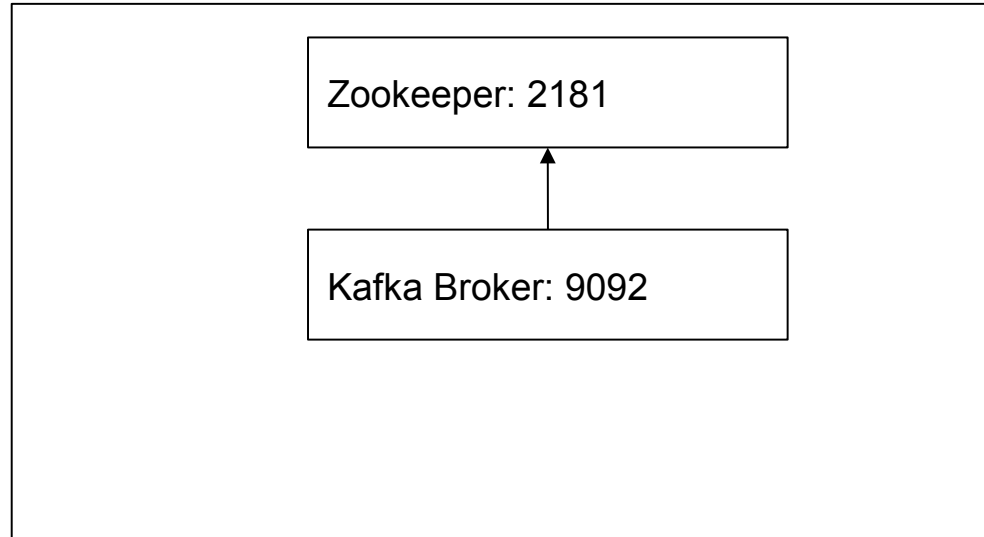
4.5

A RUNNING KAFKA SYSTEM

```
kafka-console-producer.sh|bat  
--broker-list localhost:9092 --topic demo
```

Input prompt
> Hello <Enter>

```
kafka-console-consumer.sh|bat  
--bootstrap server localhost:9092 --topic  
demo  
Blocking console  
Hello
```

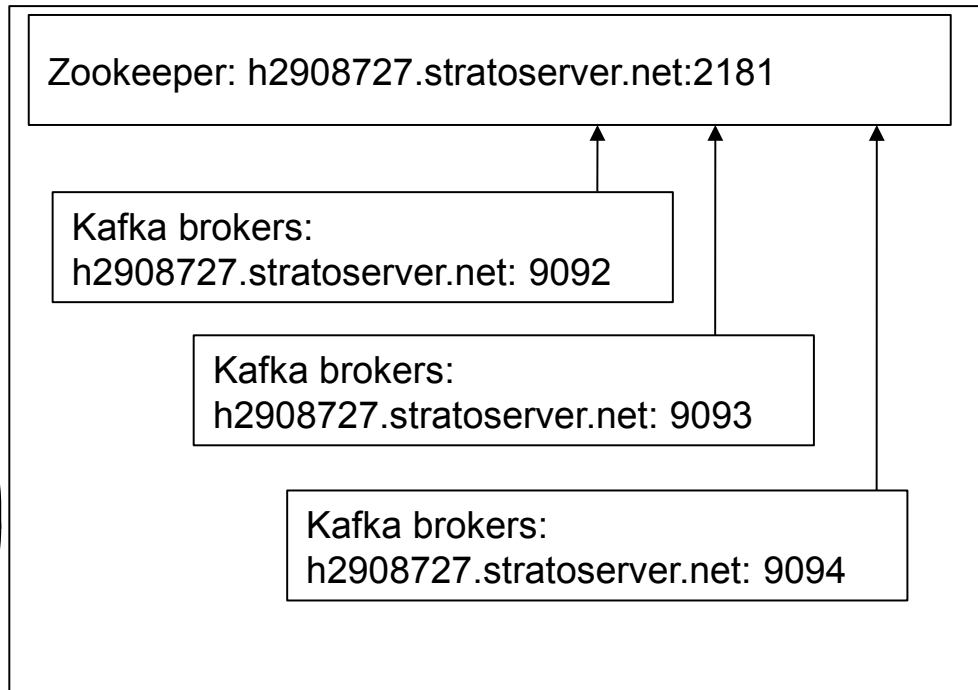


A remote Kafka cluster

```
kafka-console-producer.sh|bat
--broker-list h2908727.stratoserver.net:9092
--topic demo<name abbreviation>
```

Input prompt
> Hello <Enter>

```
kafka-console-consumer.sh|bat
--bootstrap-server
h2908727.stratoserver.net:9092 --topic demo
<name abbreviation>
Blocking console
Hello
```



5

TOPICS

5.1

TERMS

- Each topic consists of at least one partition
- Each partition is managed by exactly one broker
- Consumers read messages from a partition
- When creating a topic, the number of partitions and the number of replications can be selected
- `kafka-topics.bat -- zookeeper h2908727.stratoserver.net:2181 -- create --topic < my_topic > -- partitions 3 -- replication-factor 3`

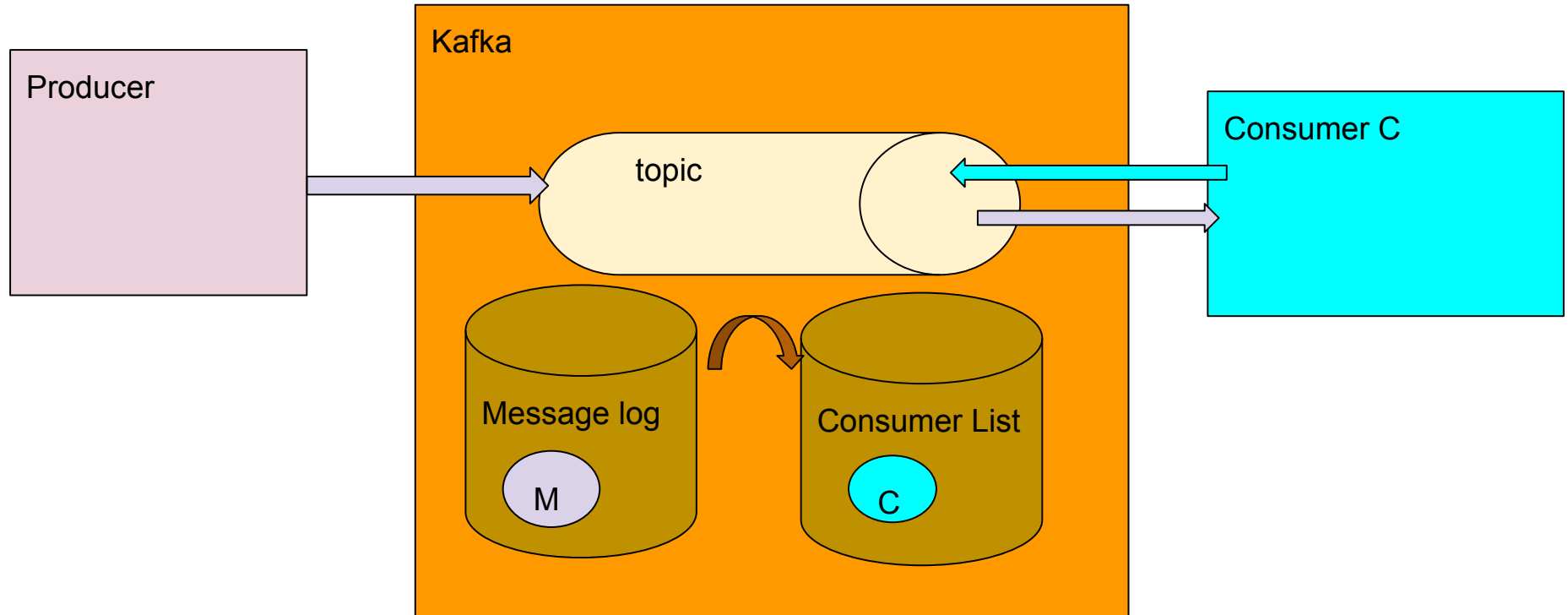
- Scalability of data from the producer's perspective
- From the perspective of consumer processes, scalability also results
- Caution:
 - Introducing new partitions is quite time-consuming (!)
 - Likewise, a rebalancing of consumers within a group

- Resulting in increased reliability
- Replication factor = 1
 - Time window in which produced data can be lost
 - Broker writes to its partition
 - and crashes
- Replication factor = 3
 - Time window in which produced data can be lost
 - Broker writes to its partition
 - Broker writes to the replication servers
 - and crashes
 - Property “Number of Acknowledgments”
 - ack = 1
 - ack = 3

- Each consumer is assigned to a consumer group
 - Identification via the `group.id`
- The commit of an offset is per consumer group
- A partition of a topic is guaranteed to only be read by one consumer of a consumer group

5.2

DETAILED WORKING METHOD



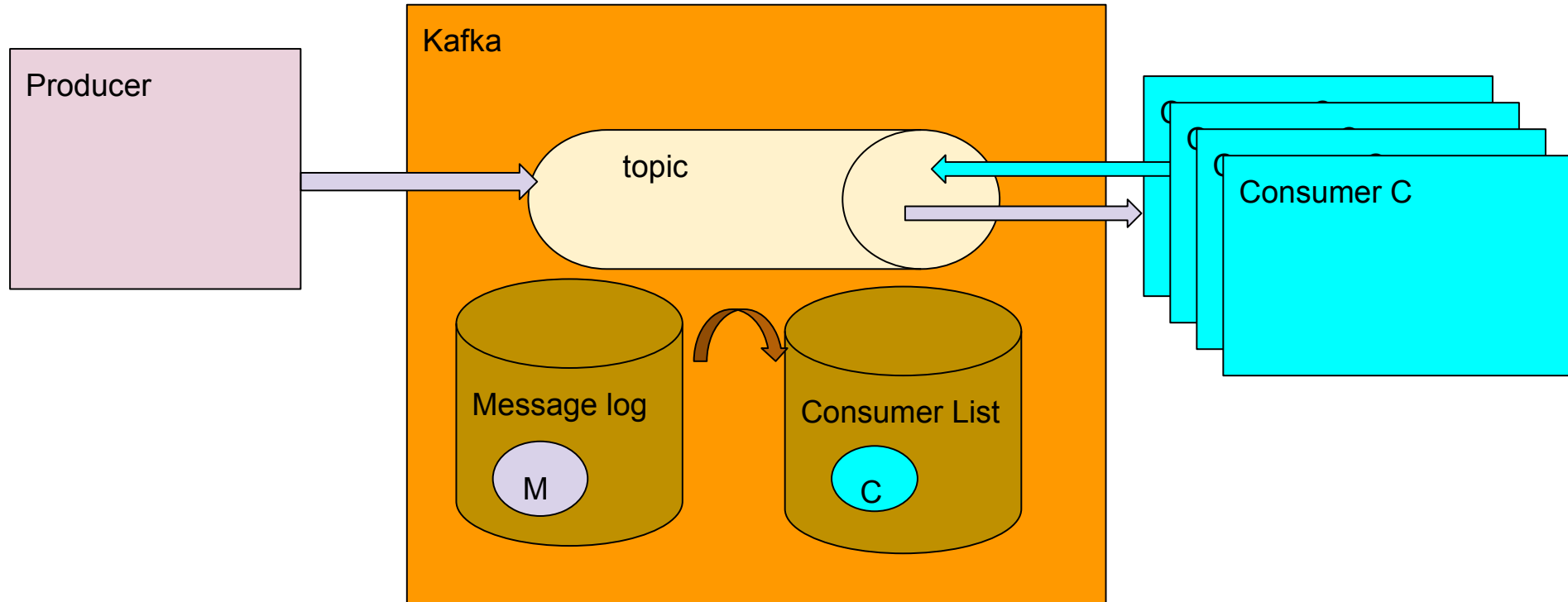
- “Kafka doesn’t delete”
- The message log is append-only
- Writing a message to the log is extremely fast
- The status of the messages that have already been delivered is recorded in the so-called “commit log”.
 - Saves the status “which messages were delivered”
 - Each consumer can receive a relative or absolute offset and thus also consume messages that have already been delivered

Kafka works as a database system

- More precisely: a key-value store
 - producer “insert into”
 - consumer “select * from where offset = ”

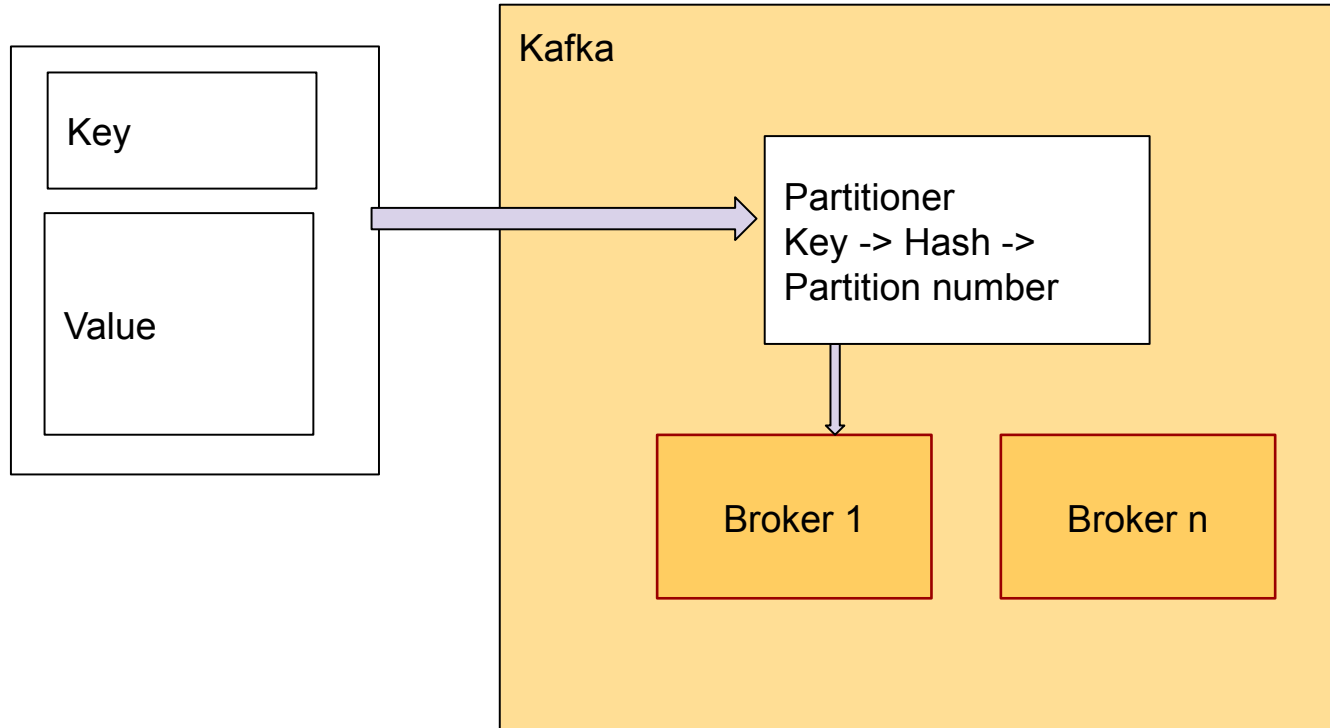
- The message log data is cleaned
 - They have a dwell time
- configuration
 - validity
 - per broker
 - per topic
 - specification
 - Size of the message log
 - Length of time
 - The criterion for deletion is the union

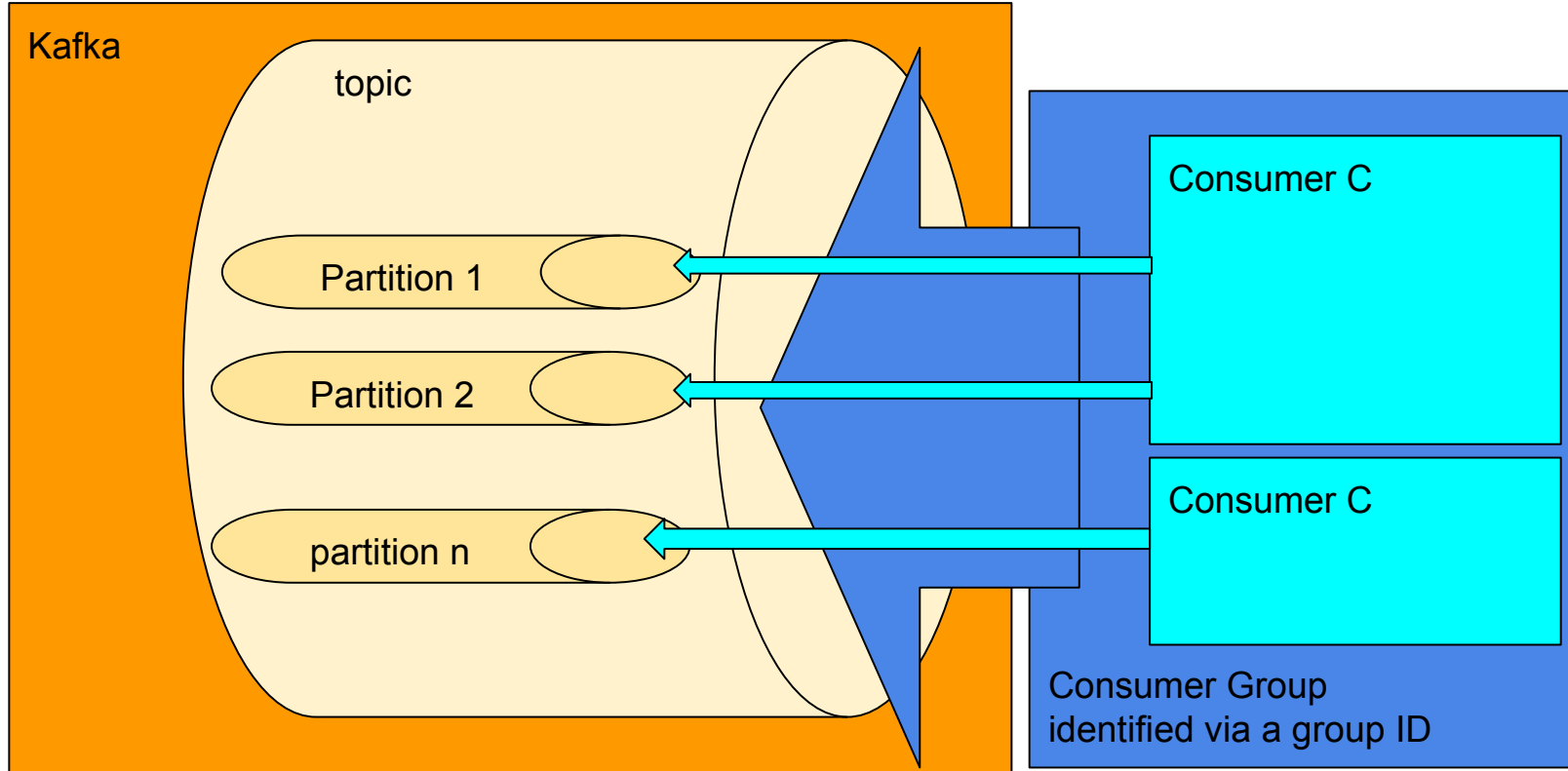
- Definition of communication channels
- Parallelization of consumer processes
 - Each topic delivers its messages to potentially multiple consumers
 - Each registered consumer manages its own commit

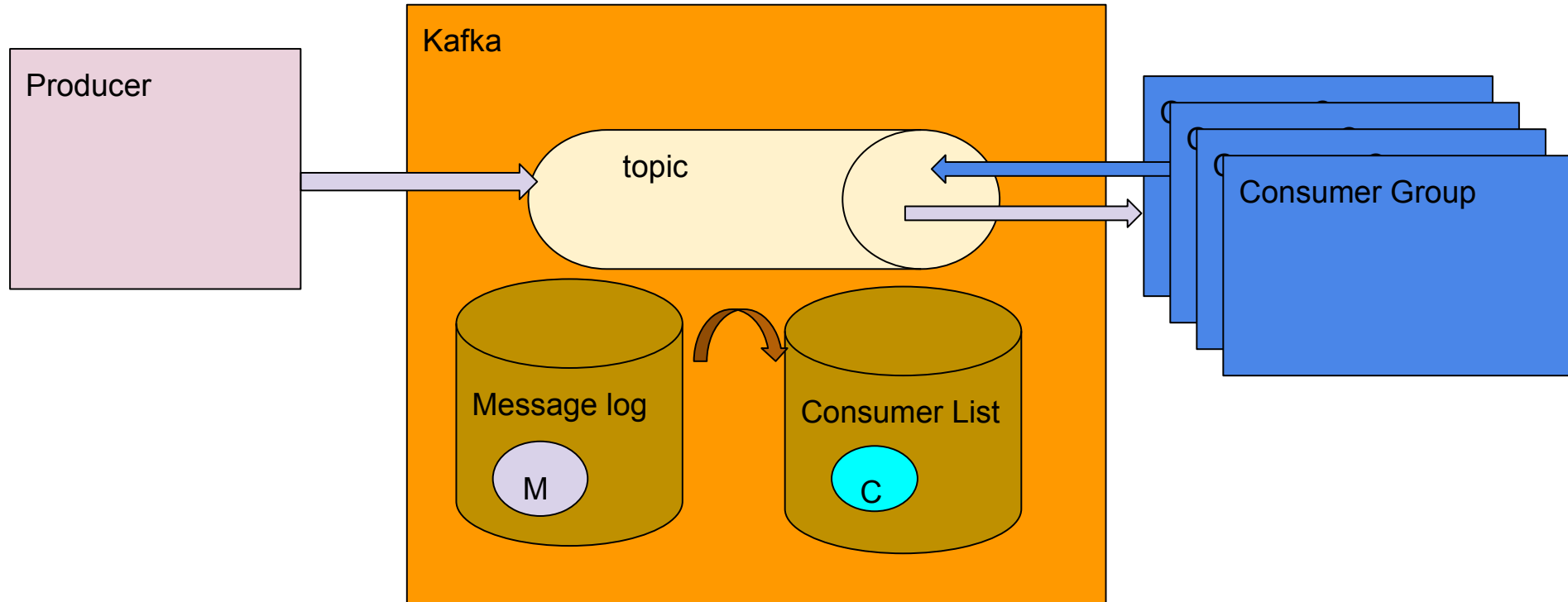


- From the consumer's perspective
 - Classic messaging: Queue
 - Each message reaches exactly one consumer
 - More than one consumer registered
 - Load distribution of the queue via Round Robin or First Available
 - Kafka does not support point-to-point messaging
- From the producer's perspective
- From the broker's perspective
 - Distributed data storage

- Partitioning the topics
- Each broker “only” holds one partition of the topic
 - This means that each topic contains at least one partition
- Caution: Changing the number of partitions of a topic is possible but time-consuming
 - The number of partitions should be determined in advance
 - It's not really dynamic
- Producers write the data to the cluster “specifying the partition”
 - by specifying a key







5.3

GUARANTEES OF NIGHT INTELLIGENCE TRANSMISSION

- Other systems
 - Classic database systems use ACID transactions
 - Atomic, Consistent, Isolated, Durable
 - Messaging systems
 - Point to Point
 - Each message is guaranteed to be received by a single consumer
 - Publish-Subscribe
 - Every message is guaranteed to be sent to all active consumers
 - Publish-Durable Subscription
 - Every message is guaranteed to be sent to all registered consumers

- **Kafka**
 - As a database system, Kafka is a realization of a BASE architecture
 - Basically Available, Soft State, Eventual (“in the end”, not “eventually”) Consistency
 - **News**
 - Anything...
 - At most once
 - Each message is processed a maximum of once
 - includes: 0
 - At least once
 - Each message is processed at least once
 - contains duplicates
 - Exactly once
 - Each message is processed exactly once

- The leader of a partition certainly guarantees consistency for itself
- `producer.send(...)`
 - Batch mode, potentially asynchronous
- Retry mechanisms
 - Network problem is a criterion for a retry
 - Duplicates are possible
- Acknowledge
 - 0, 1, all
- Transactional
 - Guaranteed message delivery without duplicates
 - A transaction log is now also written in the cluster
 - Producer ID enables the broker to detect duplicates

- Current: “Auto Commit”
 - Background process that commits at a configurable interval
 - If the consumer fails
 - Partition offset is not incremented
 - Duplicates
- Without AutoCommit
 - Synchronous commit
 - Asynchronous commit
- The consumer can also commit the message reception transactionally

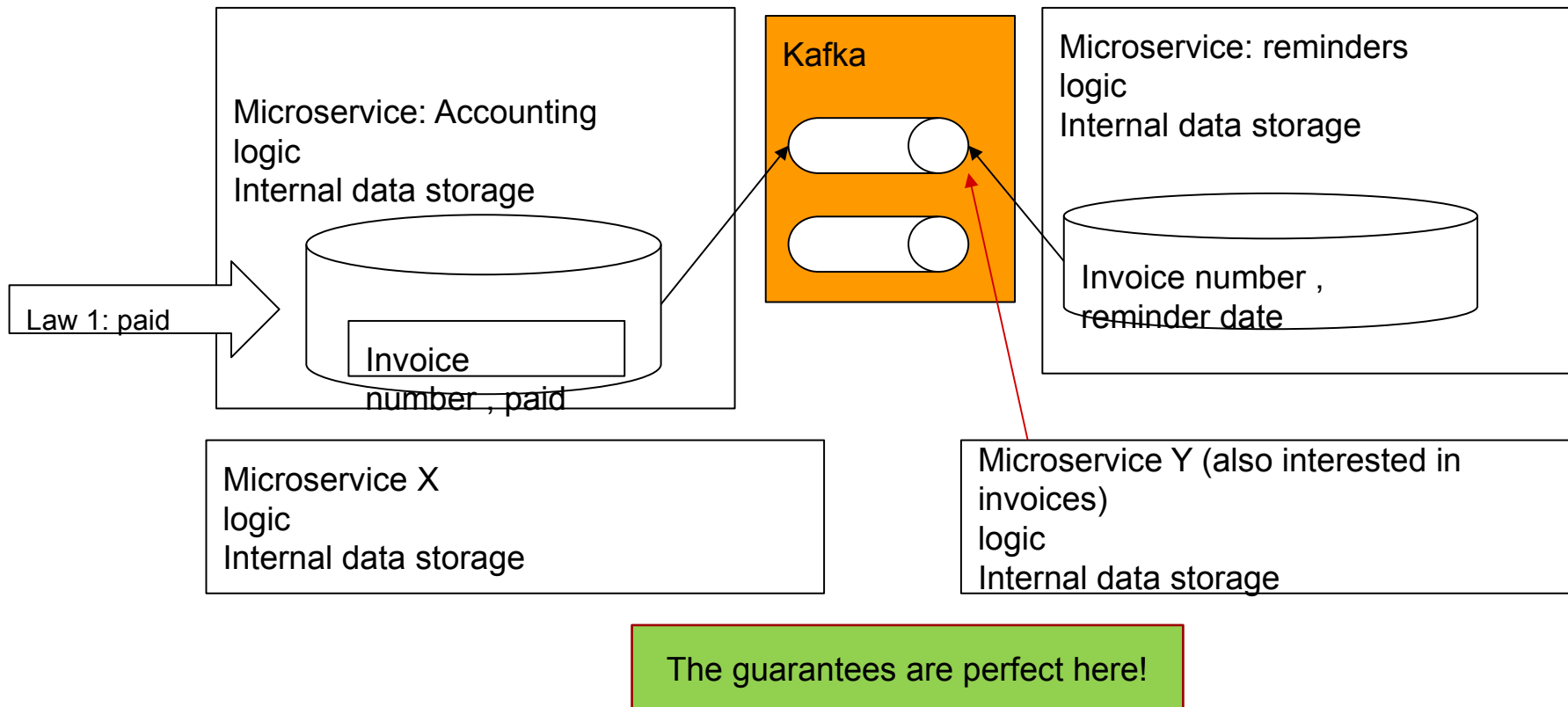
How do I do something?

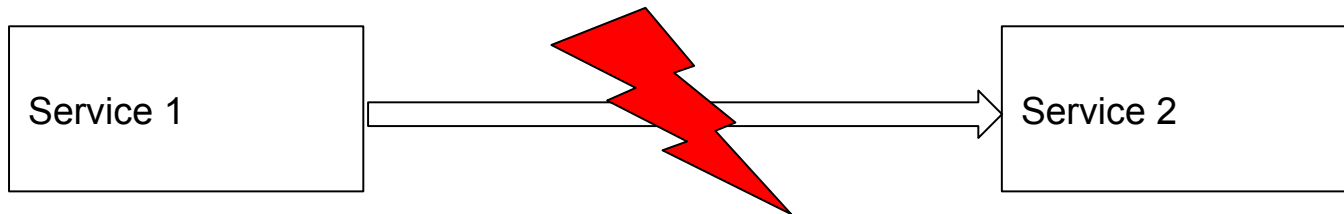
- Any
 - Kafka default
 - Reasonable default for using Kafka for loose coupling of microservices
- At Most Once
 - Producer sends synchronously with ack=all
 - Consumer
 - Disable Auto Commit
 - Synchronous commit
 - Strict: Transactional commit
- At Least Once
 - Producer sends synchronously with ack=all
 - Consumer
 - Auto commit
- Exactly once
 - Transactional consumer and producer

- Transactional
 - <https://github.com/eugenp/tutorials/blob/master/libraries-6/src/main/java/com/baeldung/kafka/TransactionalWordCount.java>
 - <https://www.baeldung.com/kafka-exactly-once>
- Message processing guarantees in Kafka
 - <https://medium.com/@andy.bryant/processing-guarantees-in-kafka-12dd2e30be0e>

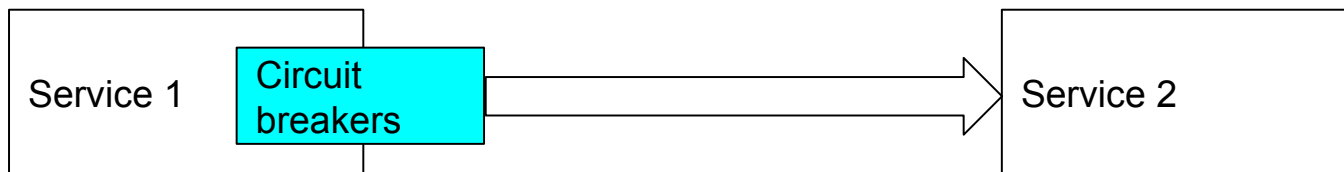
5.4

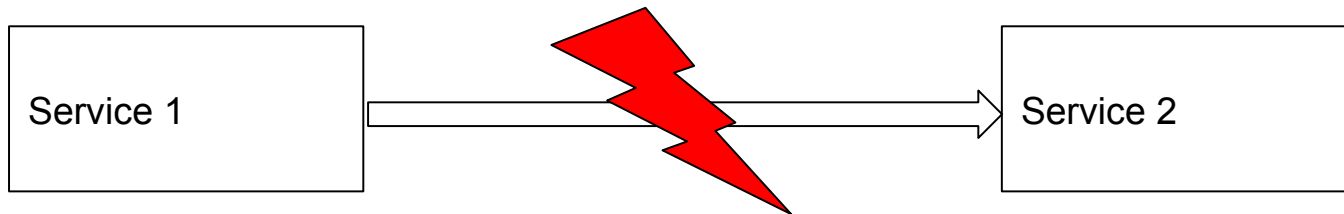
KAFKA AND SERVICES IN THE CLOUD



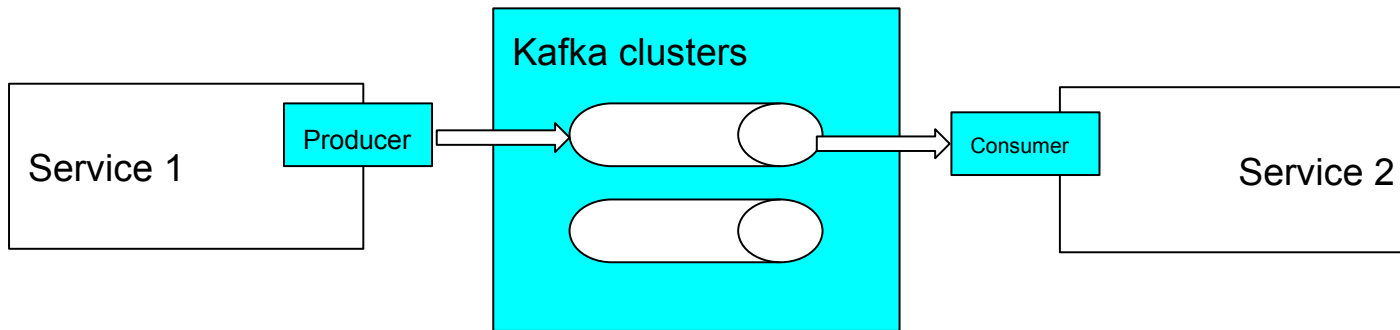


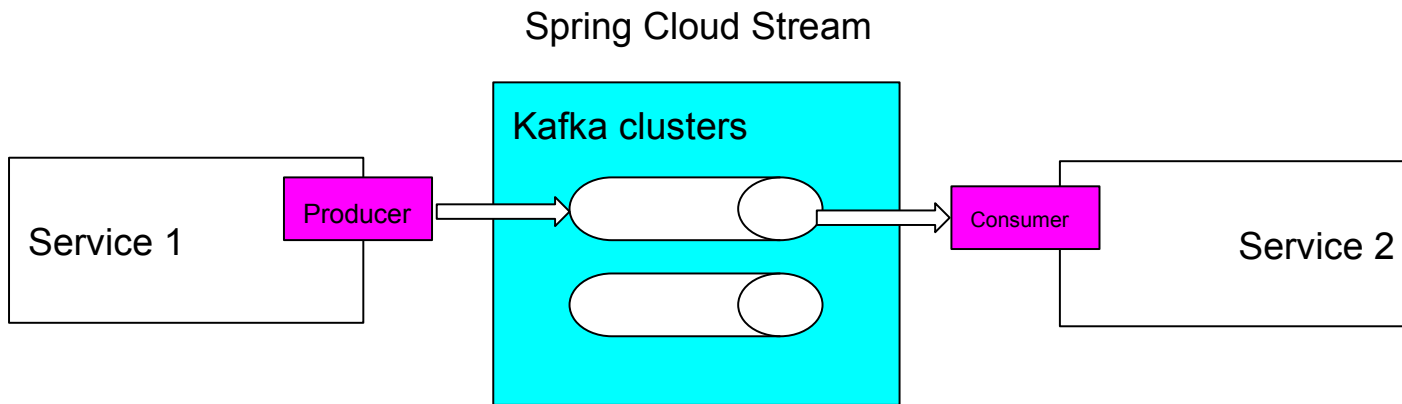
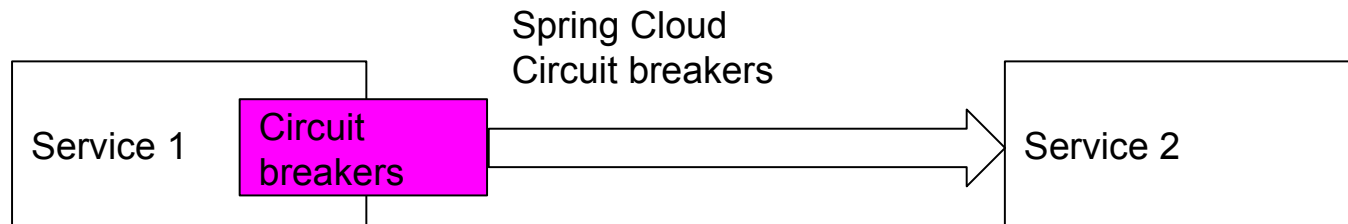
problems
What happens if there is a
“conspicuous” service 2?





problems
What happens if there is a
“conspicuous” service 2?





6

PROGRAMMING WITH KAFKA

6.1

OVERVIEW

- Consists of any number of so-called topics
 - Each topic defines a communication channel
- Development environment will generate topics itself if necessary
 - In reality, this is of course an administrative act

6.2

A FIRST PROGRAM

- Simpler Producer and a simpler Consumer are “connected” via a commonly known topic
 - <https://github.com/Javacream/org.javacream.training.kafka/tree/d23e6b80c992988b39c03172a221191841b31ac8>

- Driver libraries implement a thick client
 - Works with an initial broker list
 - The first responding broker sends back meta information about which brokers currently exist
 - Automatic retry
 - Batch mode
 - A sending process is not carried out directly, but a queue exists
 - Sending procedure
 - “Fire and Forget”
 - Synchronous or asynchronous
- Configuration settings
 - e.g. batch size, “collection time” for the batch, ..

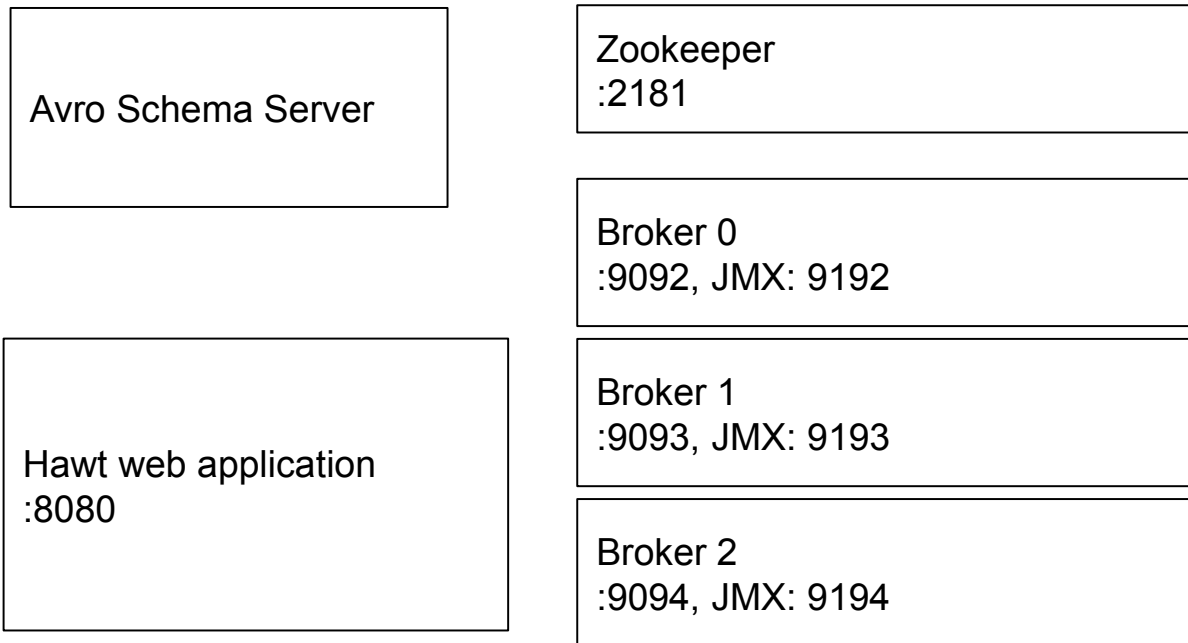
- Fetch size
- Commit
 - The offset is the “marker” that tells which messages have already been transmitted to a consumer

- Until now:
 - `topic , value`
- Well
 - `topic , key , value`
 - `key` **determines the partition (!)**
 - `kafka-run-class.bat kafka.tools.GetOffsetShell --broker-list ... --topic --time -1`

6.3

SERIALIZATION AND DESERIALIZATION

- Standard Java data types
 - `String`, `int` , ...
- Interesting for the values: JSON serializer
- Avro
 - Serialization and deserialization of your Java objects into a schema-containing document
- Custom data types
 - Two interfaces in the Kafka package
 - `Serializer <T>`
 - `Deserializer <T>`
 - Typical implementation implements both interfaces
 - “Serde ”



```
<<class>>  
LogMessage  
level:String  
message:String
```

```
<<class>>  
LogMessageKeySerde
```

```
<<class>>  
LogMessageValueSerde
```

6.4

PARTITIONER

- Determines the associated partition to be used from the key

```
interface Partitioner {  
    int partition (String topic , Object keyObject , byte [] keyRaw ,  
        Object valueObject , byte [] valueRaw ,  
Cluster kafka Cluster ){  
    partitionInfos = kafkaCluster.availablePartitionsForTopic ( topic );  
}
```

- Return value: Number of the partition to be used for this key
- Topic and partition number can be taken into account during consumer registration

7

ADMINISTRATION AND MONITORING

7.1

JMX

- A Kafka broker is a normal Java process
 - Heap memory
 - `-Xms` , `-Xmx`
 - Initial and maximum heap memory of the JVM
 - Normal Java garbage collection
 - Actually a direct contradiction to the requirements of Kafka
 - Stop-the- world effect through garbage collection, in particular: defragmentation partially using only a single CPU
 - Proportional to the maximum size of the memory
 - Duration: From milliseconds to seconds
 - Necessary: Record all garbage collections in a log file
 - `java -Xloggc:file`

- “Unexplained” drops in throughput
- Zookeeper problems
 - Frequent leader elections
 - Out of Sync/In Sync

- The `-Xloggc:file` option must be specified in the Java process call
- To evaluate this file
 - Each garbage collection exists as a line in this file
 - Timestamp, minor/major, storage before - after, duration
 - Editor, Excel, gcviewer or similar

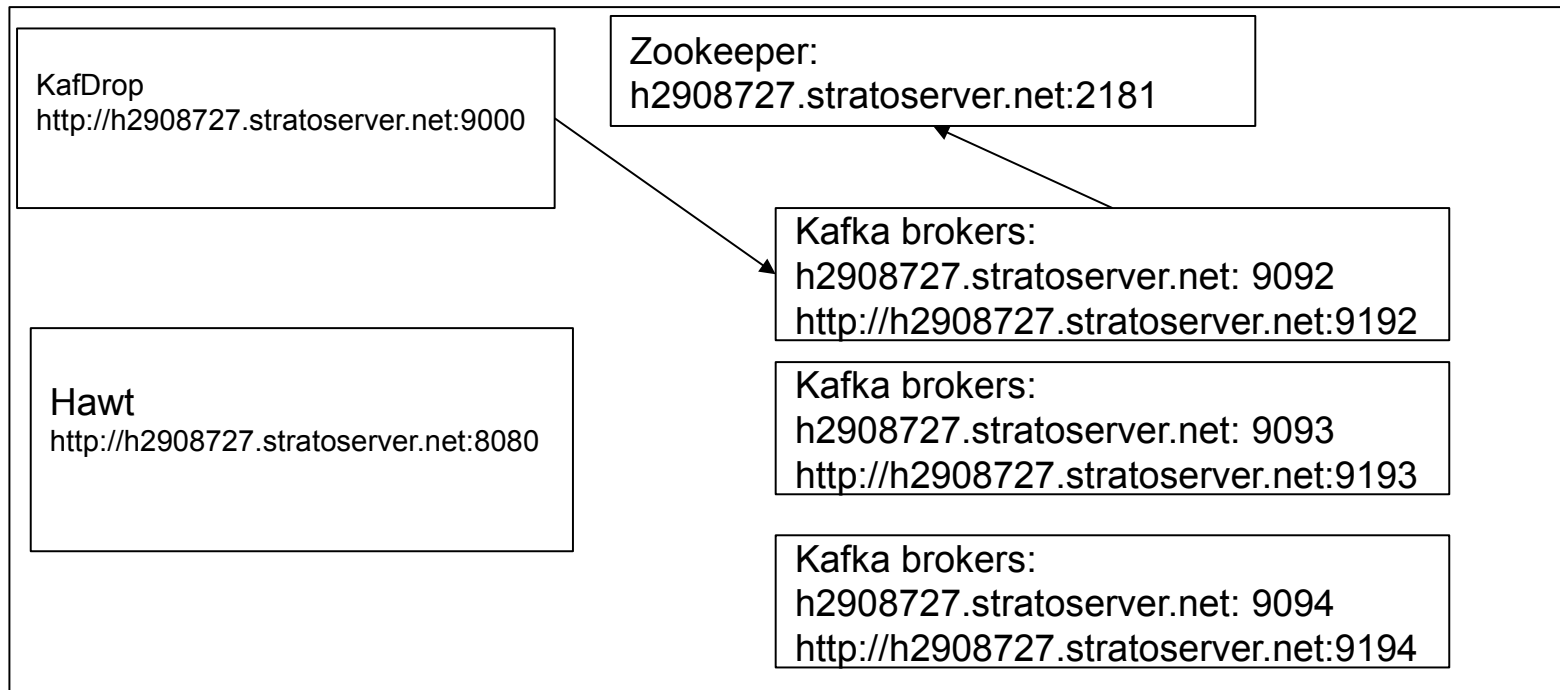
- Internally, Kafka uses the so-called “off heap memory”, “native memory”
 - Something like a “RAM disk”
 - There is no garbage collection going on here
 - Heap access time is significantly faster

- Standard JVM metrics
 - Memory, CPU etc.
 - `java.lang:type=Memory`
- Broker metrics
 - messages per second
 - Bytes per second
 - ...
- Producer and consumer metrics
 - for Java clients

- Do not use jconsole and remove the default JMX configuration in the call script
 - Actually a security problem
- Allow access to JMX with Jolokia
 - <https://java.integrata-cegos.de/jolokia-simples-management-von-java-applications/>
- Hawtio

7.2

A MONITORED CLUSTER

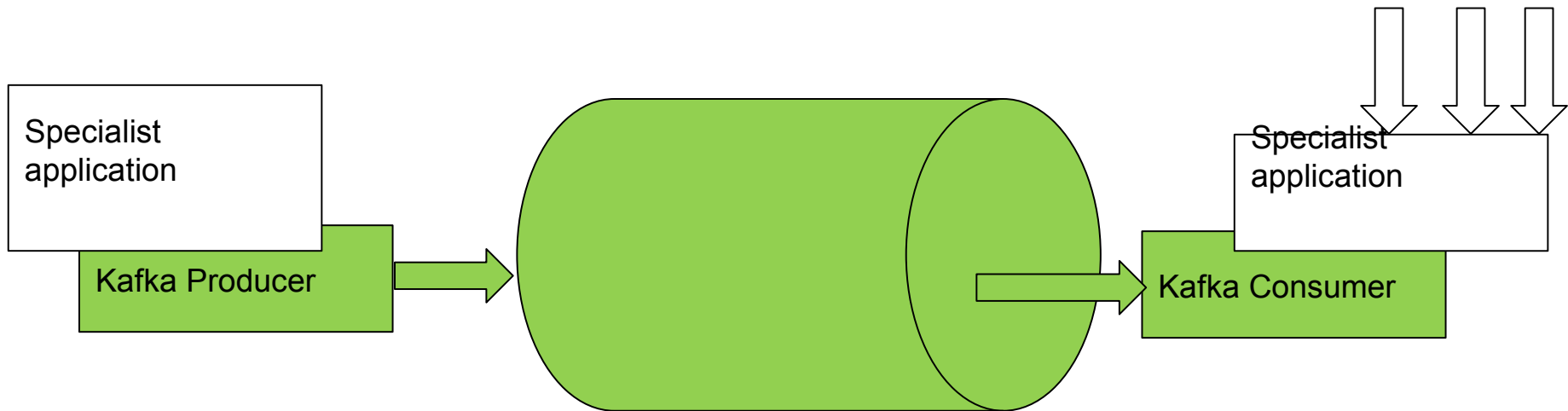


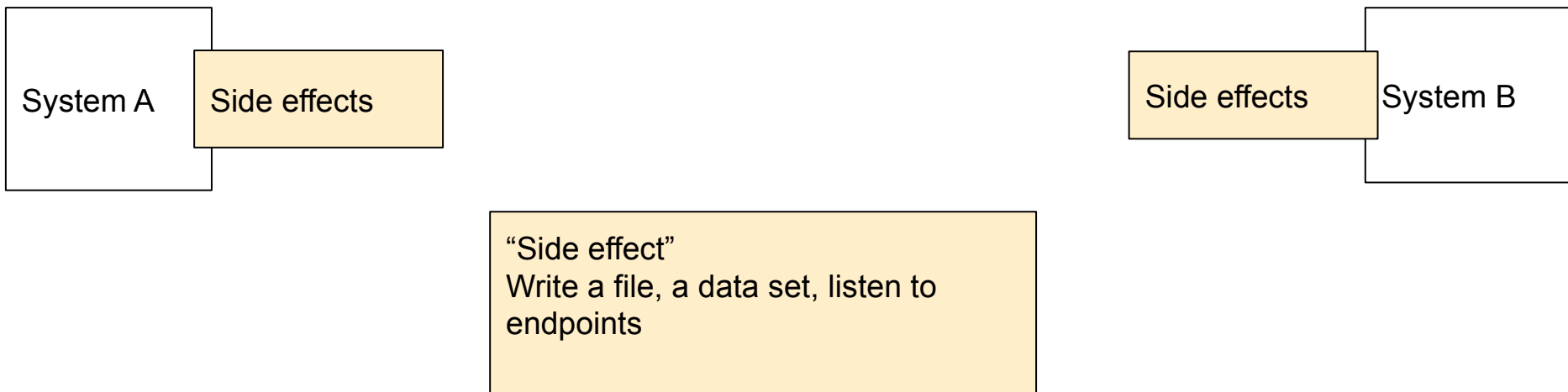
8th

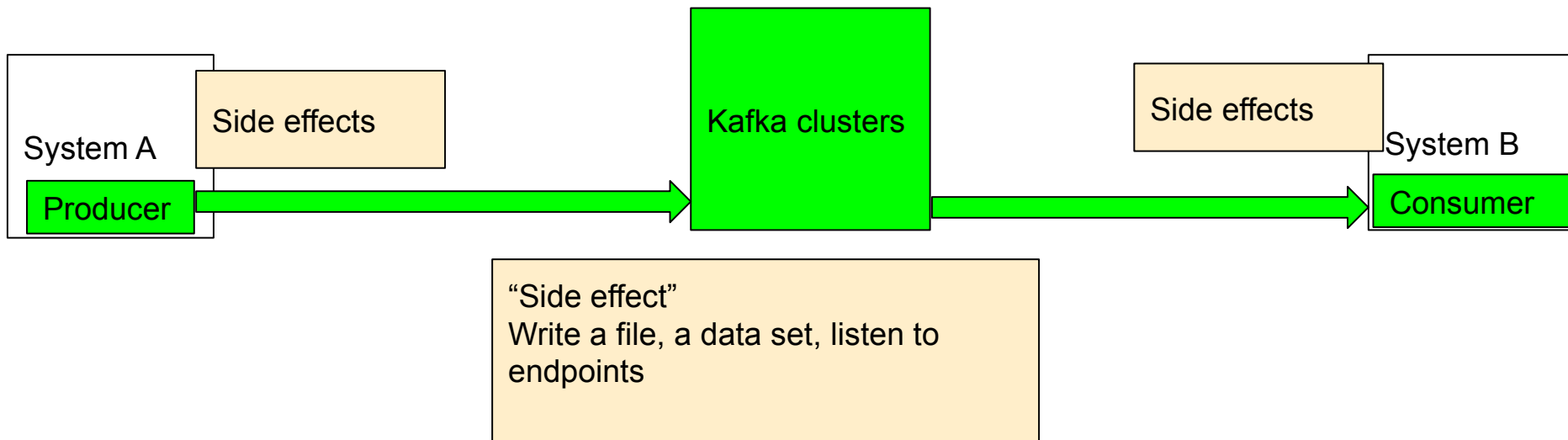
KAFKA EXTENSIONS

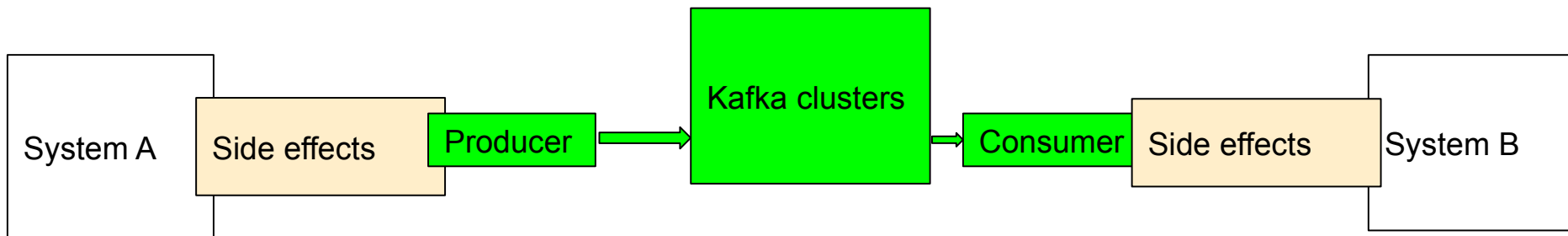
8.1

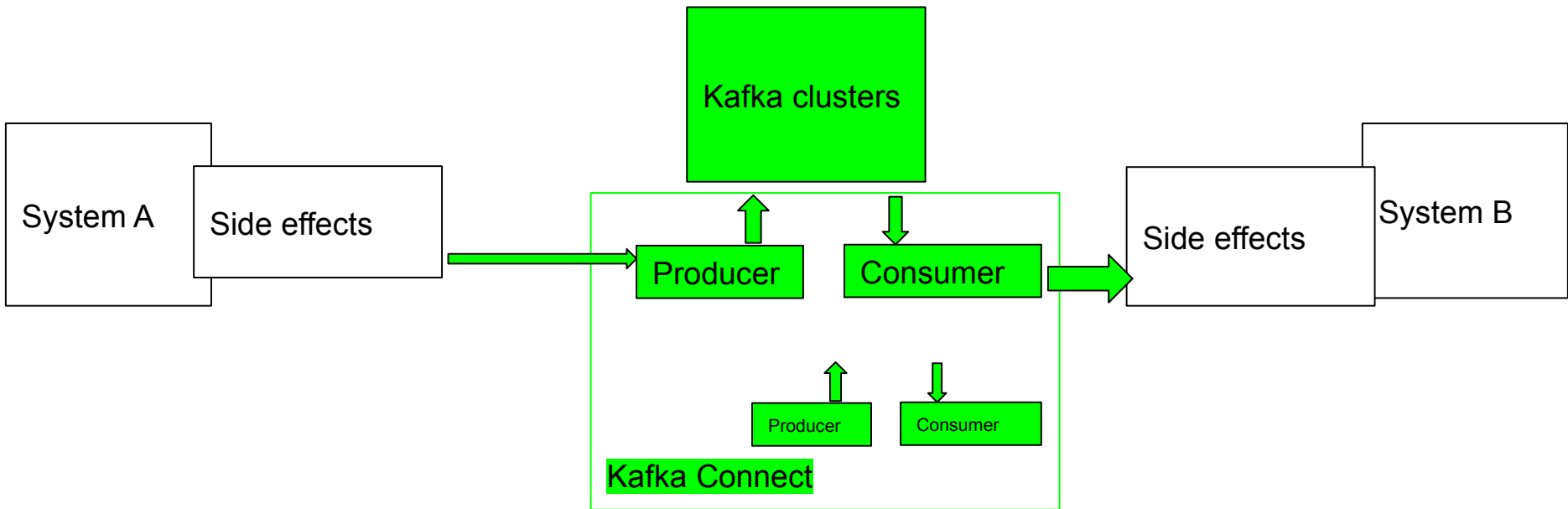
KAFKA CONNECT











- Producers and consumers
 - Separate applications
 - operated independently of the Kafka cluster
 - Connection to the cluster: configurable
 - Standalone
 - with a simple integrated Kafka broker
- Start with
 - `bin\connect-distributed.sh config\connect-distributed.properties`
 - From now on: REST endpoint on port 8083

- purely configurational
- For example, a JSON document describes a producer
- `{'name':'demo-producer', config: {'connector-class': '...FileSource'}}`
- `{'name':'demo-consumer', config: {'connector-class': '...FileSink'}}`

- Apache Camel
 - EAI framework
 - Messaging system is an internal bus
- Spring integration

8.2

KAFKA STREAM

- Special case of a streaming process
- Data processing logic
 - Data source (has no defined end)
 - Processing pipeline
 - filter
 - transform
 - aggregates
 - reduce
 - Without aggregate/reduce: forwarding to a data sink

- KStream provides a streaming API
- Operates on topics/partitions
 - permanent data stream
- filter and map operations are not critical
 - These are “Pure functions”: Parameter - function - Result
- Aggregate functions, e.g. maximum value or minimum value of a column of numbers? Average?
 - In extreme cases, the aggregation requires access to all data
 - Kafka is now an application server

- Flume or Spark
 - Internal communication and data storage can take place in Kafka